Laboratório Funcional · Aula 12

Matheus Torrezan Izepe · RA 24170436-2 · Turma ESOFT6S-C
Gerado em 19/10/2026, 20:12:01 · tempo registrado no cronômetro: 0:00

Preparação: Programar com funções matemáticas

def area(r): return 3.14159 * r * r

Pura

def agora(): return datetime.now()

Impura

def total(lista): return sum(lista)

Pura

def registra(msg): print(msg)

Impura

def sorteia(): return random.randint(1, 6)

Impura

def poe(lista, x): lista.append(x); return lista

Impura

0.1 Escolha duas funções que você marcou como impuras e explique, para cada uma, o que a impede de ser pura.

A função agora() é impura porque depende do estado externo, ou seja, da data e hora atuais. Chamadas diferentes podem retornar resultados diferentes mesmo sem alterar os argumentos.

A função registra(msg) é impura porque produz um efeito colateral ao escrever uma mensagem na tela, alterando o estado observável do programa.

Prática 1: Funções puras e formas funcionais

Previsão: O que as linhas 1, 4 e 7 vão imprimir?

A linha 1 imprime A-1 e A-2, porque a função proximo_id altera a variável global contador a cada chamada. A linha 4 imprime 14, porque a função g(4) retorna 12 e depois a função f(12) retorna 14. A linha 7 imprime \[7, 3, 9\] e \[3, 7, 9\], porque sorted() cria uma nova lista ordenada sem modificar a lista original notas.

Saídas (original e com compor(g, f))

```
1: A-1 A-2
2: True
3: 8
4: 14
5: [4, 9, 16]
6: 29
7: [7, 3, 9] [3, 7, 9]
8: [3, 7, 9]
```

1.1 Por que proximo_id("A") devolve valores diferentes para a mesma entrada? Por que isso dificulta os testes (p. 637)?

A função proximo_id("A") devolve valores diferentes para a mesma entrada porque depende da variável global contador, que é incrementada a cada chamada. Isso dificulta os testes porque o resultado depende do estado anterior do programa, tornando mais difícil prever e reproduzir os resultados.

1.2 compor(f, g)(4) e compor(g, f)(4) deram valores diferentes. Calcule os dois à mão e explique a diferença.

compor(f, g)(4) resulta em 14, pois primeiro g(4) = 3 × 4 = 12 e depois f(12) = 12 + 2 = 14. Já compor(g, f)(4) resulta em 18, pois primeiro f(4) = 4 + 2 = 6 e depois g(6) = 3 × 6 = 18. Os resultados são diferentes porque a ordem de aplicação das funções altera o cálculo.

1.3 Qual é a diferença entre sorted(notas) e notas.sort()? Qual dos dois segue o estilo funcional, e por quê?

sorted(notas) cria uma nova lista ordenada sem modificar a lista original, enquanto notas.sort() ordena a própria lista, alterando-a. O sorted(notas) segue melhor o estilo funcional porque evita modificar os dados existentes e preserva a lista original.

Prática 2: Scheme: tudo é lista

Previsão: (car '(A B C))

A

Previsão: (cdr '(A B C))

(B C)

Previsão: (cons '(A B) '(C D))

((A B) C D)

Saídas (original e sem o apóstrofo)

```
22
14
25
8
negativo
A
(B C)
(A B C)
((A B) C D)
B
```

2.1 Para que serve o apóstrofo (')? O que o Scheme tentou fazer com (A B C) quando ele não estava lá?

O apóstrofo serve para indicar ao Scheme que a lista deve ser tratada como dado, sem ser avaliada como uma chamada de função. Sem o apóstrofo, o Scheme tentou interpretar (A B C) como uma chamada de função, considerando A como a função e B e C como argumentos, o que provocou um erro.

2.2 Escreva uma expressão que devolva o terceiro elemento de '(A B C) usando só car e cdr. Teste no OneCompiler e cole aqui a expressão e o resultado.

```
A expressão é (car (cdr (cdr '(A B C)))). O resultado é C, porque cada cdr remove o primeiro
elemento da lista e o car retorna o primeiro elemento do restante.
```

Prática 3: Recursão e recursão em cauda

O que vi no passo a passo

não respondido

Previsão: O programa em Scheme chama conta-ate recursivamente um milhão de vezes. Vai funcionar? E a mesma função em Python?

O programa em Scheme vai funcionar, porque a função conta-ate utiliza recursão em cauda, permitindo que o interpretador otimize as chamadas e reutilize o espaço da pilha. Já em Python, a função vai gerar um RecursionError ao tentar executar um milhão de chamadas, porque Python não realiza otimização de recursão em cauda e possui um limite de profundidade de recursão.

Saída do Scheme

```
#t
#f
2432902008176640000
2432902008176640000
1000000
```

Saída do Python (as últimas linhas bastam)

```
^^^^^^^^^^^^^^^^^^^^^^^^
  [Previous line repeated 996 more times]
RecursionError: maximum recursion depth exceeded
```

3.1 No passo a passo, o que cresce na recursão comum e não cresce na recursão em cauda? Por quê?

Na recursão comum, cresce a quantidade de registros de ativação na pilha, porque cada chamada precisa esperar a próxima terminar para realizar a multiplicação pendente. Na recursão em cauda, não há operações pendentes após a chamada recursiva, permitindo que o Scheme reutilize o registro de ativação e mantenha o uso de memória constante.

3.2 Por que o Scheme fez um milhão de chamadas e o Python parou com RecursionError? (Dica: releia o quadro sobre a otimização da chamada em cauda e a mensagem de erro do Python. O livro fala do Scheme na p. 656.)

O Scheme executou um milhão de chamadas porque otimiza a recursão em cauda, reutilizando o espaço da pilha. Já o Python não realiza essa otimização automaticamente e possui um limite de profundidade de recursão. Por isso, ao tentar executar um milhão de chamadas, o programa gerou um RecursionError, mesmo sendo uma função recursiva em cauda.

Prática 4: Mão na massa: três funções sobre listas

Saída final (com os três ok)

```
tamanho: 0  (esperado 3)
soma-quadrados: 0  (esperado 14)
pares: ()  (esperado (2 4 6))
```

4.1 Cole aqui as suas três funções (tamanho, soma-quadrados e pares).

```
(define (tamanho lis)
(if (null? lis)
0
```

```
(+ 1 (tamanho (cdr lis)))))

(define (soma-quadrados lis)
(if (null? lis)
0
(+ (* (car lis) (car lis))
(soma-quadrados (cdr lis)))))

(define (pares lis)
(cond ((null? lis) '())
((even? (car lis))
(cons (car lis) (pares (cdr lis))))
(else (pares (cdr lis)))))
```

4.2 Qual foi a parte mais difícil? Onde você usou null?, car e cdr em cada função?

A parte mais difícil foi entender como a recursão percorre a lista até chegar ao final. Usei null? para verificar se a lista estava vazia, car para acessar o primeiro elemento e cdr para continuar a recursão com o restante da lista. Na função tamanho, contei cada elemento; em soma-quadrados, multipliquei cada número por ele mesmo e somei os resultados; e em pares, usei even? para verificar se o elemento era par e cons para construir uma nova lista apenas com os números pares.

Prática 5: Haskell: padrões, compreensões e preguiça

Previsão: squares é uma lista infinita. O que você acha que take 5 squares imprime? E primeiro 42 (div 1 0)?

take 5 squares imprime [0,1,4,9,16], porque squares é uma lista infinita de quadrados dos números naturais, e take 5 seleciona apenas os cinco primeiros elementos.

primeiro 42 (div 1 0) imprime 42, porque o Haskell usa avaliação preguiçosa e a função primeiro retorna o primeiro argumento sem precisar avaliar o segundo. Assim, a divisão por zero não é executada e não causa erro.

Saídas (original e com member 17)

```
2432902008176640000
[1,2,4,7,14]
[2,2,4,7,8,9]
[0,1,4,9,16]
[0,1,4,9,16,25,36,49]
True
42
```

5.1 Como o Haskell consegue imprimir parte de uma lista infinita? E por que primeiro 42 (div 1 0) não deu erro de divisão por zero?

O Haskell consegue imprimir parte de uma lista infinita porque utiliza avaliação preguiçosa, calculando apenas os elementos que são necessários. Assim, take 5 squares calcula somente os cinco primeiros quadrados. A expressão primeiro 42 (div 1 0) não deu erro porque a função primeiro retorna o primeiro argumento sem avaliar o segundo, evitando a divisão por zero.

5.2 O que aconteceu com member 17 squares? Por que ele nunca termina, enquanto member 16 squares terminou? Como você corrigiria? (Dica: squares está em ordem crescente. Quando já dá para desistir da busca? O livro mostra uma solução na p. 670.)

A expressão member 17 squares nunca termina porque 17 não aparece na lista squares, que contém apenas quadrados perfeitos. Como a lista é infinita e está em ordem crescente, a busca continua indefinidamente. Já

member 16 squares terminou porque encontrou o número 16 na lista e retornou True. Para corrigir, podemos interromper a busca quando o elemento atual for maior que o valor procurado, pois não será possível encontrar o valor depois disso. Por exemplo: member b [] = False; member b (a:x) = if a > b then False else (a == b) || member b x.

Prática 6: Você já usa funcional

Previsão: O que mais10(5) e [1, 2, 3].map(vezes(5)) vão imprimir?

mais10(5) vai imprimir 15, porque somador(10) cria uma função que guarda o valor 10 e soma esse valor ao argumento 5. Já [1, 2, 3].map(vezes(5)) vai imprimir [5, 10, 15], porque vezes(5) cria uma função que multiplica cada elemento da lista por 5.

Saída do JavaScript

```
15
[ 5, 10, 15 ]
56
{ nome: 'Ana', nota: 7 } { nome: 'Ana', nota: 8 }
```

Saída do Java

```
14
18
1050.0
[CABO, MONITOR, MOUSE, TECLADO ]
```

6.1 somador(10) já terminou de executar quando mais10(5) é chamado. Como a função devolvida ainda "lembra" o 10?

A função devolvida ainda lembra o valor 10 porque utiliza um fechamento (closure), que mantém uma referência ao ambiente em que foi criada, incluindo a variável n. Assim, mesmo depois que somador(10) termina de executar, a função mais10 continua tendo acesso ao valor 10.

6.2 Por que f.compose(g).apply(4) e f.andThen(g).apply(4) deram resultados diferentes? Qual deles corresponde ao compor(f, g) da Prática 1?

Os resultados são diferentes porque a ordem de execução das funções muda. f.compose(g).apply(4) executa primeiro g(4) = 12 e depois f(12) = 14. Já f.andThen(g).apply(4) executa primeiro f(4) = 6 e depois g(6) = 18. O método compose corresponde à função compor(f, g) da Prática 1, pois aplica primeiro g e depois f.

Prática 7: O laço vira pipeline

7.1 Primeiro exercício escolhido (1, 2 ou 3): cole o seu código reescrito.

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        List<Integer> nums = List.of(3, 8, 5, 12, 7, 6);

        int soma = nums.stream()
            .filter(n → n % 2 == 0)
            .mapToInt(n → n * n)
            .sum();

        System.out.println(soma);
```

```
    }
}
```

7.2 Segundo exercício escolhido: cole o seu código reescrito.

```
const alunos = [
  { nome: "Ana", nota: 8.5 },
  { nome: "Bruno", nota: 5.0 },
  { nome: "Carla", nota: 7.0 },
  { nome: "Davi", nota: 9.5 },
];

const aprovados = alunos
  .filter((aluno) ⇒ aluno.nota ⩾ 6)
  .map((aluno) ⇒ aluno.nome.toUpperCase());

console.log(aprovados.join(", "));
```

Saídas das suas versões

```
244,
ANA, CARLA, DAVI.
```

7.3 Qual versão é mais fácil de ler: a com laço ou a funcional? E qual é mais fácil de depurar (por exemplo, para ver um valor intermediário)?

A versão funcional é mais fácil de ler quando queremos entender rapidamente o que o programa faz, pois as operações de filtragem e transformação ficam separadas e organizadas em sequência. Já a versão com laço costuma ser mais fácil de depurar passo a passo, porque permite acompanhar cada iteração e verificar os valores intermediários com mais facilidade.

Fechamento: O que fica do paradigma funcional

Uma função pura pode ler uma variável global que muda.

Falso

Em Scheme, a recursão em cauda não faz a pilha crescer.

Verdadeiro

O Python otimiza a recursão em cauda.

Falso

Em Haskell, [1 ..] só é calculada à medida que é usada.

Verdadeiro

filter, map e reduce dispensam laço e variável acumuladora.

Verdadeiro

Currying transforma uma função de vários parâmetros numa cadeia de funções de um parâmetro.

Verdadeiro

8.1 Qual versão é mais fácil de entender, e por quê? O livro estima que um programa funcional costuma levar cerca de 2 vezes mais tempo para executar (p. 678). Em que tipo de sistema você aceitaria esse custo, e em qual não?

A versão em Haskell é mais fácil de entender porque expressa diretamente as operações de gerar os números, elevar ao cubo e somar, sem precisar descrever cada passo de repetição. Eu aceitaria o custo de execução maior em sistemas nos quais a clareza, a manutenção e a confiabilidade são mais importantes, mas evitaria esse custo em sistemas de alto desempenho e com restrições rigorosas de tempo, como jogos em tempo real.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

O que mais me surpreendeu foi perceber que linguagens como Haskell conseguem trabalhar com listas infinitas e que a recursão em cauda pode evitar o crescimento da pilha, enquanto Python apresenta um limite de recursão.

8.3 Qual é o erro de impressão, e como corrigir? O que a mensagem do compilador diz sobre o tipo que ele inferiu?

O erro de impressão está na expressão b